

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



MICROPROCESSOR MICRONTROLLER (CO3009)

LAB 4

A COOPERATIVE SCHEDULER

Class: L04 – Semester HK251
Submission Date:14/11/2025

Instructor: Mr. Ton Huynh Long

Student Name	Student ID
Phạm Công Võ	2313946

Ho Chi Minh City, September 2025



Table of Contents

1	Introduction	3
1.1	Super Loop Architecture	3
1.2	Timer-based interrupts and interrupt service routines	4
2	What is a scheduler?	5
2.1	The co-operative scheduler	5
2.2	Function pointers	6
2.3	Solution	7
2.3.1	Overview	7
2.3.2	The scheduler data structure and task array	9
2.3.3	The initialization function	9
2.3.4	The ‘Update’ function	10
2.3.5	The ‘Add Task’ function	11
2.3.6	The ‘Dispatcher’	12
2.3.7	The ‘Delete Task’ function	14
2.3.8	Reducing power consumption	14
2.3.9	Reporting errors	15
2.3.10	Adding a watchdog	16
2.3.11	Reliability and safety implications	17
2.3.12	Portability	18
3	Objectives	18
4	Problem	18
5	Demonstration	18
6	Submission	19
7	Transforming SCH_Update to O(1)	19
7.1	SCH_Init	21
7.2	SCH_Add_Task	22
7.3	SCH_Delete_Task	27
7.4	SCH_Update	29
7.5	SCH_Dispatch_Tasks	31
	References	36



List of Figures

1 Introduction

1.1 Super Loop Architecture

```
1 }  
2 void main(void){  
3     // Prepare for Task X  
4     X_Init();  
5     while(1) { // 'for ever' (Super Loop)  
6         X(); // Perform the task  
7     }  
8 }
```

The main advantages of the Super Loop architecture illustrated above are:

- (1) that it is simple, and therefore easy to understand, and
- (2) that it consumes virtually no system memory or CPU resources.

However, we get ‘nothing for nothing’: Super Loops consume little memory or processor resources because they provide few facilities to the developer. A particular limitation with this architecture is that it is very difficult to execute Task X at precise intervals of time: as we will see, this is a very significant drawback.

For example, consider a collection of requirements assembled from a range of different embedded projects (in no particular order):

- The current speed of the vehicle must be measured at 0.5 second intervals.
- The display must be refreshed 40 times every second.
- The calculated new throttle setting must be applied every 0.5 seconds.
- A time-frequency transform must be performed 20 times every second.
- If the alarm sounds, it must be switched off (for legal reasons) after 20 minutes.
- If the front door is opened, the alarm must sound in 30 seconds if the correct password is not entered in this time.
- The engine vibration data must be sampled 1,000 times per second.
- The frequency-domain data must be classified 20 times every second.
- The keypad must be scanned every 200 ms.
- The master (control) node must communicate with all other nodes (sensor nodes and sounder nodes) once per second.
- The new throttle setting must be calculated every 0.5 seconds.
- The sensors must be sampled once per second.

We can summarize this list by saying that many embedded systems must carry out tasks at particular instants of time. More specifically, we have two kinds of activity to perform:

- Periodic tasks, to be performed (say) once every 100 ms
- One-shot tasks, to be performed once after a delay of (say) 50 ms

This is very difficult to achieve with the primitive architecture shown in Program above. Suppose, for example, that we need to start Task X every 200 ms, and that the task takes 10 ms to complete. Program below illustrates one way in which we might adapt the code in order to try to achieve this.

```
1 }  
2 void main(void){  
3     // Prepare for Task X  
4     X_Init();  
5     while(1) {           // 'for ever' (Super Loop)  
6         X();              // Perform the task (10 ms duration)  
7         Delay_190ms();    // Delay for 190 ms  
8     }  
9 }
```

The approach is not generally adequate, because it will only work if the following conditions are satisfied:

- We know the precise duration of Task X
- This duration never varies

In practical applications, determining the precise task duration is rarely straightforward. Suppose we have a very simple task that does not interact with the outside world but, instead, performs some internal calculations. Even under these rather restricted circumstances, changes to compiler optimization settings – even changes to an apparently unrelated part of the program – can alter the speed at which the task executes. This can make fine-tuning the timing very tedious and error prone.

The second condition is even more problematic. Often in an embedded system the task will be required to interact with the outside world in a complex way. In these circumstances the task duration will vary according to outside activities in a manner over which the programmer has very little control.

1.2 Timer-based interrupts and interrupt service routines

A better solution to the problems outlined is to use timer-based interrupts as a means of invoking functions at particular times.

An interrupt is a hardware mechanism used to notify a processor that an ‘event’ has taken place: such events may be internal events or external events.

When an interrupt is generated, the processor ‘jumps’ to an address at the bottom of the CODE memory area. These locations must contain suitable code with which the microcontroller can respond to the interrupt or, more commonly, the locations will include another ‘jump’ instruction, giving the address of suitable ‘interrupt service routine’ located elsewhere in (CODE) memory.

Please see lab 3 for the more information of this approach.

2 What is a scheduler?

There are two ways of viewing a scheduler:

- At one level, a scheduler can be viewed as a simple operating system that allows tasks to be called periodically or (less commonly) on a one-shot basis.
- At a lower level, a scheduler can be viewed as a single timer interrupt service routine that is shared between many different tasks. As a result, only one timer needs to be initialized, and any changes to the timing generally requires only one function to be altered. Furthermore, we can generally use the same scheduler whether we need to execute one, ten or 100 different tasks.

```
1 void main(void) {  
2     // Set up the scheduler  
3     SCH_Init();  
4     // Add the tasks (1ms tick interval)  
5     // Function_A will run every 2 ms  
6     SCH_Add_Task(Function_A, 0, 2);  
7     // Function_B will run every 10 ms  
8     SCH_Add_Task(Function_B, 1, 10);  
9     // Function_C will run every 15 ms  
10    SCH_Add_Task(Function_C, 3, 15);  
11    while(1) {  
12        SCH_Dispatch_Tasks();  
13    }  
14 }
```

2.1 The co-operative scheduler

A co-operative scheduler provides a single-tasking system architecture

Operation:

- Tasks are scheduled to run at specific times (either on a periodic or one-shot basis)
- When a task is scheduled to run it is added to the waiting list
- When the CPU is free, the next waiting task (if any) is executed
- The task runs to completion, then returns control to the scheduler

Implementation:

- The scheduler is simple and can be implemented in a small amount of code
- The scheduler must allocate memory for only a single task at a time
- The scheduler will generally be written entirely in a high-level language (such as 'C')

- The scheduler is not a separate application; it becomes part of the developer's code

Performance:

- Obtaining rapid responses to external events requires care at the design stage Reliability and safety:

Co-operate scheduling is simple, predictable, reliable and safe

A co-operative scheduler provides a simple, highly predictable environment. The scheduler is written entirely in 'C' and becomes part of the application: this tends to make the operation of the whole system more transparent and eases development, maintenance and porting to different environments. Memory overheads are 17 bytes per task and CPU requirements (which vary with tick interval) are low.

2.2 Function pointers

One area of the language with which many 'C' programmers are unfamiliar is the function pointer. While comparatively rarely used in desktop programs, this language feature is crucial in the creation of schedulers: we therefore provide a brief introductory example here.

The key point to note is that – just as we can, for example, determine the starting address of an array of data in memory – we can also find the address in memory at which the executable code for a particular function begins. This address can be used as a 'pointer' to the function; most importantly, it can be used to call the function. Used with care, function pointers can make it easier to design and implement complex programs. For example, suppose we are developing a large, safety-critical, application, controlling an industrial plant. If we detect a critical situation, we may wish to shut down the system as rapidly as possible. However, the appropriate way to shut down the system will vary, depending on the system state. What we can do is create a number of different recovery functions and a function pointer. Every time the system state changes, we can alter the function pointer so that it is always pointing to the most appropriate recovery function. In this way, we know that – if there is ever an emergency situation – we can rapidly call the most appropriate function, by means of the function pointer.

```
1 // ----- Private function prototypes -----
2 void Square_Number(int, int*);
3
4 int main(void)
5 {
6     int a = 2, b = 3;
7     /* Declares pFn to be a pointer to fn with
8      int and int pointer parameters (returning void) */
9     void (* pFn)(int, int*);
10
11     int Result_a, Result_b;
12     pFn = Square_Number; // pFn holds address of Square_Number
13     printf("Function code starts at address: %u\n", (tWord) pFn);
14     printf("Data item a starts at address: %u\n\n", (tWord) &a);
15     // Call 'Square_Number' in the conventional way
```

```
16     Square_Number(a, &Result_a);
17     // Call 'Square_Number' using function pointer
18     (*pFn)(b, &Result_b);
19     printf("%d squared is %d (using normal fn call)\n", a, Result_a)
20     ;
21     printf("%d squared is %d (using fn pointer)\n", b, Result_b);
22     while(1);
23     return 0;
24 }
25 void Square_Number(int a, int* b)
26 { // Demo - calculate square of a
27     *b = a * a;
28 }
```

2.3 Solution

A scheduler has the following key components:

- The scheduler data structure.
- An initialization function.
- A single interrupt service routine (ISR), used to update the scheduler at regular time intervals.
- A function for adding tasks to the scheduler.
- A dispatcher function that causes tasks to be executed when they are due to run.
- A function for removing tasks from the scheduler (not required in all applications).

We consider each of the required components in this section

2.3.1 Overview

Before discussing the scheduler components, we consider how the scheduler will typically appear to the user. To do this we will use a simple example: a scheduler used to flash a single LED on and off repeatedly: on for one second off for one second etc.

```
1 int main(void){
2     //Init all the requirments for the system to run
3     System_Initialization();
4     //Init a schedule
5     SCH_Init();
6     //Add a task to repeatedly call in every 1 second.
```



```
7 SCH_Add_Task(Led_Display, 0, 1000);  
8 while (1){  
9     SCH_Dispatch_Tasks();  
10 }  
11 return 0;  
12 }
```

- We assume that the LED will be switched on and off by means of a ‘task’ `Led_Display()`. Thus, if the LED is initially off and we call `Led_Display()` twice, we assume that the LED will be switched on and then switched off again.

To obtain the required flash rate, we therefore require that the scheduler calls `Led_Display()` every second ad infinitum.

- We prepare the scheduler using the function `SCH_Init()`.
- After preparing the scheduler, we add the function `Led_Display()` to the scheduler task list using the `SCH_Add_Task()` function. At the same time we specify that the LED will be turned on and off at the required rate as follows:

`SCH_Add_Task(Led_Display, 0, 1000);`

We will shortly consider all the parameters of `SCH_Add_Task()`, and examine its internal structure.

- The timing of the `Led_Display()` function will be controlled by the function `SCH_Update()`, an interrupt service routine triggered by the overflow of Timer 2:

```
1 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){  
2     SCH_Update();  
3 }
```

- The ‘Update’ function does not execute the task: it calculates when a task is due to run and sets a flag. The job of executing `LED_Display()` falls to the dispatcher function (`SCH_Dispatch_Tasks()`), which runs in the main (‘super’) loop:

```
1 while(1){  
2     SCH_Dispatch_Tasks();  
3 }
```

Before considering these components in detail, we should acknowledge that this is, undoubtedly, a complicated way of flashing an LED: if our intention were to develop an LED flasher application that requires minimal memory and minimal code size, this would not be a good solution. However, the key point is that we will be able to use the same scheduler architecture in all our subsequent examples, including a number of substantial and complex applications and the effort required to understand the operation of this environment will be rapidly repaid.

It should also be emphasized that the scheduler is a ‘low-cost’ option: it consumes a small percentage of the CPU resources (we will consider precise percentages shortly). In addition, the scheduler itself requires no more than 17 bytes of memory for each task. Since a typical application will require no more

than four to six tasks, the task – memory budget (around 60 bytes) is not excessive, even on an 8-bit microcontroller.

2.3.2 The scheduler data structure and task array

At the heart of the scheduler is the scheduler data structure: this is a user-defined data type which collects together the information required about each task.

```
1
2 typedef struct {
3     // Pointer to the task (must be a 'void (void)' function)
4     void ( * pTask)(void);
5     // Delay (ticks) until the function will (next) be run
6     uint32_t Delay;
7     // Interval (ticks) between subsequent runs.
8     uint32_t Period;
9     // Incremented (by scheduler) when task is due to execute
10    uint8_t RunMe;
11    //This is a hint to solve the question below.
12    uint32_t TaskID;
13 } sTask;
14
15 // MUST BE ADJUSTED FOR EACH NEW PROJECT
16 #define SCH_MAX_TASKS      40
17 #define NO_TASK_ID         0
18 sTask SCH_tasks_G[SCH_MAX_TASKS];
```

The size of the task array

You must ensure that the task array is sufficiently large to store the tasks required in your application, by adjusting the value of SCH_MAX_TASKS. For example, if you schedule three tasks as follows:

- SCH_Add_Task(Function_A, 0, 2);
- SCH_Add_Task(Function_B, 1, 10);
- SCH_Add_Task(Function_C, 3, 15);

then SCH_MAX_TASKS must have a value of three (or more) for correct operation of the scheduler. Note also that, if this condition is not satisfied, the scheduler should generate an error code.

2.3.3 The initialization function

Like most of the tasks we wish to schedule, the scheduler itself requires an initialization function. While this performs various important operations – such as preparing the scheduler array (discussed earlier) and the error code variable (discussed later) – the main purpose of this function is to set up a timer that will be used to generate the regular ‘ticks’ that will drive the scheduler.

```
1 void SCH_Init(void) {
2     unsigned char i;
3     for (i = 0; i < SCH_MAX_TASKS; i++) {
4         SCH_Delete_Task(i);
5     }
6     // Reset the global error variable
7     // - SCH_Delete_Task() will generate an error code,
8     // (because the task array is empty)
9     Error_code_G = 0;
10    Timer_init();
11    Watchdog_init();
12 }
```

2.3.4 The 'Update' function

The 'Update' function is involved in the ISR. It is invoked when the timer is overflow.

When it determines that a task is due to run, the update function increments the RunMe field for this task: the task will then be executed by the dispatcher, as we discuss later.

```
1 void SCH_Update(void){
2     unsigned char Index;
3     // NOTE: calculations are in *TICKS* (not milliseconds)
4     for (Index = 0; Index < SCH_MAX_TASKS; Index++) {
5         // Check if there is a task at this location
6         if (SCH_tasks_G[Index].pTask){
7             if (SCH_tasks_G[Index].Delay == 0) {
8                 // The task is due to run
9                 // Inc. the 'RunMe' flag
10                SCH_tasks_G[Index].RunMe += 1;
11                if (SCH_tasks_G[Index].Period) {
12                    // Schedule periodic tasks to run again
13                    SCH_tasks_G[Index].Delay = SCH_tasks_G[Index].
14                    Period;
15                }
16            } else {
17                // Not yet ready to run: just decrement the delay
18                SCH_tasks_G[Index].Delay -= 1;
19            }
20        }
21    }
22 }
```

```
20     }
21 }
22
23 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
24     SCH_Update();
25 }
```

2.3.5 The ‘Add Task’ function

As the name suggests, the ‘Add Task’ function is used to add tasks to the task array, to ensure that they are called at the required time(s). Here is the example of add task function: **unsigned char SCH_Add_Task (Task_Name , Initial_Delay, Period)**

The parameters for the ‘Add Task’ function are described as follows:

- **Task_Name**: the name of the function (task) that you wish to schedule
- **Initial_Delay**: the delay (in ticks) before task is first executed. If set to 0, the task is executed immediately.
- **Period**: the interval (in ticks) between repeated executions of the task. If set to 0, the task is executed only once

Here are some examples.

This set of parameters causes the function Do_X() to be executed once after 1,000 scheduler ticks:

SCH_Add_Task(Do_X,1000,0);

This does the same, but saves the task ID (the position in the task array) so that the task may be subsequently deleted, if necessary (see SCH_Delete_Task() for further information about the removal of tasks from the task array):

Task_ID = SCH_Add_Task(Do_X,1000,0);

This causes the function Do_X() to be executed regularly every 1,000 scheduler ticks; the task will first be executed as soon as the scheduling is started:

SCH_Add_Task(Do_X,0,1000);

This causes the function Do_X() to be executed regularly every 1,000 scheduler ticks; task will be first executed at T = 300 ticks, then 1,300, 2,300 etc:

SCH_Add_Task(Do_X,300,1000);

```
1  /*
   -----*
2  SCH_Add_Task() Causes a task (function) to be executed at regular
   intervals
3  or after a user-defined delay
4  -*-----
   */
```

```
5 unsigned char SCH_Add_Task(void (* pFunction)(), unsigned int DELAY,
   unsigned int PERIOD)
6 {
7     unsigned char Index = 0;
8     // First find a gap in the array (if there is one)
9     while ((SCH_tasks_G[Index].pTask != 0) && (Index < SCH_MAX_TASKS
10 ))
11     {
12         Index++;
13     }
14     // Have we reached the end of the list?
15     if (Index == SCH_MAX_TASKS)
16     {
17         // Task list is full
18         // Set the global error variable
19         Error_code_G = ERROR_SCH_TOO_MANY_TASKS;
20         // Also return an error code
21         return SCH_MAX_TASKS;
22     }
23     // If we're here, there is a space in the task array
24     SCH_tasks_G[Index].pTask = pFunction;
25     SCH_tasks_G[Index].Delay = DELAY;
26     SCH_tasks_G[Index].Period = PERIOD;
27     SCH_tasks_G[Index].RunMe = 0;
28     // return position of task (to allow later deletion)
29     return Index;
30 }
```

2.3.6 The 'Dispatcher'

As we have seen, the 'Update' function does not execute any tasks: the tasks that are due to run are invoked through the 'Dispatcher' function.

```
1 void SCH_Dispatch_Tasks(void)
2 {
3     unsigned char Index;
4     // Dispatches (runs) the next task (if one is ready)
5     for (Index = 0; Index < SCH_MAX_TASKS; Index++){
```

```
6         if (SCH_tasks_G[Index].RunMe > 0) {
7             (*SCH_tasks_G[Index].pTask)(); // Run the task
8             SCH_tasks_G[Index].RunMe -= 1; // Reset / reduce RunMe
9             flag
10             // Periodic tasks will automatically run again
11             // - if this is a 'one shot' task, remove it from the
12             array
13             if (SCH_tasks_G[Index].Period == 0)
14             {
15                 SCH_Delete_Task(Index);
16             }
17         }
18         // Report system status
19         SCH_Report_Status();
20         // The scheduler enters idle mode at this point
21         SCH_Go_To_Sleep();
22     }
```

The dispatcher is the only component in the Super Loop:

```
1 void main(void)
2 {
3     ...
4     while(1)
5     {
6         SCH_Dispatch_Tasks();
7     }
```

Do we need a Dispatch function?

At first inspection, the use of both the 'Update' and 'Dispatch' functions may seem a rather complicated way of running the tasks. Specifically, it may appear that the Dispatch function is unnecessary and that the Update function could invoke the tasks directly. However, the split between the Update and Dispatch operations is necessary, to maximize the reliability of the scheduler in the presence of long tasks.

Suppose we have a scheduler with a tick interval of 1 ms and, for whatever reason, a scheduled task sometimes has a duration of 3 ms.

If the Update function runs the functions directly then – all the time the long task is being executed – the tick interrupts are effectively disabled. Specifically, two 'ticks' will be missed. This will mean that all system timing is seriously affected and may mean that two (or more) tasks are not scheduled to execute at all.

If the Update and Dispatch function are separated, system ticks can still be processed while the long task is executing. This means that we will suffer task ‘jitter’ (the ‘missing’ tasks will not be run at the correct time), but these tasks will, eventually, run.

2.3.7 The ‘Delete Task’ function

When tasks are added to the task array, SCH_Add_Task() returns the position in the task array at which the task has been added: Task_ID = SCH_Add_Task(Do_X,1000,0);

Sometimes it can be necessary to delete tasks from the array. To do so, SCH_Delete_Task() can be used as follows: SCH_Delete_Task(Task_ID)

```
1  /*-----  
    */  
2  unsigned char SCH_Delete_Task(const tByte TASK_INDEX){  
3      unsigned char Return_code;  
4      if (SCH_tasks_G[TASK_INDEX].pTask == 0) {  
5          // No task at this location...  
6          //  
7          // Set the global error variable  
8          Error_code_G = ERROR_SCH_CANNOT_DELETE_TASK  
9  
10         // ...also return an error code  
11         Return_code = RETURN_ERROR;  
12     } else {  
13         Return_code = RETURN_NORMAL;  
14     }  
15     SCH_tasks_G[TASK_INDEX].pTask = 0x0000;  
16     SCH_tasks_G[TASK_INDEX].Delay = 0;  
17     SCH_tasks_G[TASK_INDEX].Period = 0;  
18     SCH_tasks_G[TASK_INDEX].RunMe = 0;  
19     return Return_code; // return status  
20 }
```

2.3.8 Reducing power consumption

An important feature of scheduled applications is that they can lend themselves to low-power operation. This is possible because all modern MCU provide an ‘idle’ mode, where the CPU activity is halted, but the state of the processor is maintained. In this mode, the power required to run the processor is typically reduced by around 50

This idle mode is particularly effective in scheduled applications because it may be entered under software control, and the MCU returns to the normal operating mode when any interrupt is received.

Because the scheduler generates regular timer interrupts as a matter of course, we can put the system ‘to sleep’ at the end of every dispatcher call: it will then wake up when the next timer tick occurs.

This is an optional feature. Students can do by yourself by looking at the reference manual of the MCU that is used.

```
1 void SCH_Go_To_Sleep(){
2     //todo: Optional
3 }
```

2.3.9 Reporting errors

Hardware fails; software is never perfect; errors are a fact of life. To report errors at any part of the scheduled application, we can use an (8-bit) error code variable `Error_code_G`

```
unsigned char Error_code_G = 0;
```

To record an error we include lines such as:

- `Error_code_G = ERROR_SCH_TOO_MANY_TASKS;`
- `Error_code_G = ERROR_SCH_WAITING_FOR_SLAVE_TO_ACK;`
- `Error_code_G = ERROR_SCH_WAITING_FOR_START_COMMAND_FROM_MASTER;`
- `Error_code_G = ERROR_SCH_ONE_OR_MORE_SLAVES_DID_NOT_START;`
- `Error_code_G = ERROR_SCH_LOST_SLAVE;`
- `Error_code_G = ERROR_SCH_CAN_BUS_ERROR;`
- `Error_code_G = ERROR_I2C_WRITE_BYTE_AT24C64;`

To report these error codes, the scheduler has a function `SCH_Report_Status()`, which is called from the Update function.

```
1 void SCH_Report_Status(void) {
2     #ifdef SCH_REPORT_ERRORS
3         // ONLY APPLIES IF WE ARE REPORTING ERRORS
4         // Check for a new error code
5         if (Error_code_G != Last_error_code_G) {
6             // Negative logic on LEDs assumed
7             Error_port = 255 - Error_code_G;
8             Last_error_code_G = Error_code_G;
9             if (Error_code_G != 0){
10                 Error_tick_count_G = 60000;
11             } else {
12                 Error_tick_count_G = 0;
13             }
14         } else {
```



```
15     if (Error_tick_count_G != 0){
16         if (--Error_tick_count_G == 0)    {
17             Error_code_G = 0; // Reset error code
18         }
19     }
20 }
21 #endif
22 }
```

Note that error reporting may be disabled via the main.h header file:

```
1 // Comment this line out if error reporting is NOT required
2 // #define SCH_REPORT_ERRORS
3 // Where error reporting is required, the port on which error codes
   will be displayed
4 // is also determined via main.h:
5 #ifdef SCH_REPORT_ERRORS
6 // The port on which error codes will be displayed
7 // ONLY USED IF ERRORS ARE REPORTED
8 #define Error_port PORTA
9 #endif
```

Note that, in this implementation, error codes are reported for 60,000 ticks (1 minute at a 1 ms tick rate). The simplest way of displaying these codes is to attach eight LEDs (with suitable buffers) to the error port, as discussed in IC DRIVER [page 134]: Figure 14.3 illustrates one possible approach.

What does that error code mean? The forms of error reporting discussed here are low-level in nature and are primarily intended to assist the developer of the application or a qualified service engineer performing system maintenance. An additional user interface may also be required in your application to notify the user of errors, in a more user-friendly manner.

2.3.10 Adding a watchdog

The basic scheduler presented here does not provide support for a watchdog timer. Such support can be useful and is easily added, as follows:

- Start the watchdog in the scheduler Start function.
- Refresh the watchdog in the scheduler Update function.

```
1 IWDG_HandleTypeDef hiwdg;
2 static uint32_t counter_for_watchdog = 0;
3
4 void MX_IWDG_Init(void){
5     hiwdg.Instance = IWDG;
```

```
6  hiwdg.Init.Prescaler = IWDG_PRESCALER_32;
7  hiwdg.Init.Reload = 4095;
8  if (HAL_IWDG_Init(&hiwdg) != HAL_OK) {
9      Error_Handler();
10 }
11 }
12 void Watchdog_Refresh(void){
13     HAL_IWDG_Refresh(&hiwdg);
14 }
15 unsigned char Is_Watchdog_Reset(void){
16     if(counter_for_watchdog > 3){
17         return 1;
18     }
19     return 0;
20 }
21 void Watchdog_Counting(void){
22     counter_for_watchdog++;
23 }
24
25 void Reset_Watchdog_Counting(void){
26     counter_for_watchdog = 0;
27 }
```

2.3.11 Reliability and safety implications

- Make sure the task array is large enough
- Take care with function pointers
- Dealing with task overlap

Suppose we have two tasks in our application (Task A, Task B). We further assume that Task A is to run every second and Task B every three seconds. We assume also that each task has a duration of around 0.5 ms.

Suppose we schedule the tasks as follows (assuming a 1ms tick interval):

```
SCH_Add_Task(TaskA, 0, 1000);
SCH_Add_Task(TaskB, 0, 3000);
```

In this case, the two tasks will sometimes be due to execute at the same time. On these occasions, both tasks will run, but Task B will always execute after Task A. This will mean that if Task A varies in duration, then Task B will suffer from ‘jitter’: it will not be called at the correct time when the tasks overlap.

Alternatively, suppose we schedule the tasks as follows:

```
SCH_Add_Task(TaskA, 0, 1000);  
SCH_Add_Task(TaskB, 5, 3000);
```

Now, both tasks still run every 1,000 ms and 3,000 ms (respectively), as required. However, Task A is explicitly scheduled always to run 5 ms before Task B. As a result, Task B will always run on time.

In many cases, we can avoid all (or most) task overlaps simply by the judicious use of the initial task delays.

2.3.12 Portability

3 Objectives

The aim of this lab is to design and implement a cooperate scheduler to accurately provide timeouts and trigger activities. You should add a file for the scheduler implementation and modify the main system call loop to handle timer interrupts.

4 Problem

Your system should have at least four functions:

- **void SCH_Update(void):** This function will be updated the remaining time of each tasks that are added to a queue. It will be called in the interrupt timer, for example 10 ms.
- **void SCH_Dispatch_Tasks(void):** This function will get the task in the queue to run.
- **uint32_t SCH_Add_Task(void (* pFunction)(), uint32_t DELAY, uint32_t PERIOD):** This function is used to add a task to the queue. It should return an ID that is corresponding with the added task.
- **uint8_t SCH_Delete_Task(uint32_t taskID):** This function is used to delete the task based on its ID.

You should add more functions if you think it will help you to solve this problem. Your main program must have 5 tasks running periodically in 0.5 second, 1 second, 1.5 seconds, 2 seconds, 2.5 seconds.

5 Demonstration

You should be able to show some test code that uses all the functions specified in the driver interface. Specifically set up and demonstrate:

- A regular 10ms timer tick.
- Register a timeout to fire a callback every 10ms.
- Then, print the value returned by get_time every time this callback is received.
- Note: Your timestamps must be at least accurate to the nearest 10ms.

- Register another timeout at a different interval in addition to the 500ms running concurrently (i.e. demo more than one timeout registered at a time).
- Before entering the main loop, set up a few calls to `SCH_Add_Task`. Make sure the delay used is long enough such that the loop is entered before these wake up. These callbacks should just print out the current timestamp as each delay expires.

Note this is not a complete list. The following designs are considered unsatisfactory:

- Only supporting a single timeout registered at a time.
- Delivering callbacks in the wrong order
- $O(n)$ searches in the `SCH_Update` function.
- Interrupt frequencies greater than 10Hz, if your timer ticks regularly.

6 Submission

You need to :

- Demonstrate your work in the lab class and then
- The complete LAB4 project source code is available at: [GitHub - A COOPERATIVE SCHEDULER..](#)

7 Transforming `SCH_Update` to $O(1)$

This section aims to redesign several functions in the scheduler so that the `SCH_Update()` routine executes in $O(1)$ time complexity — meaning it only operates on a single fixed task each cycle — while keeping the `sTask` data structure unchanged.

The functions that must be modified include:

- `SCH_Init()`
- `SCH_Add_Task()`
- `SCH_Delete_Task()`
- `SCH_Update()`
- `SCH_Dispatch_Tasks()`

Main Idea

In the traditional scheduler, the `SCH_Update()` function iterates through the entire task list and decreases the `Delay` value for each task - leading to $O(n)$ time complexity.

In the new design, only the task at the beginning of the list (index 0) is updated each timer tick (e.g., every 10 ms). This reduces the complexity of `SCH_Update()` to $O(1)$, but it also requires rethinking how the list is maintained and how task delays are managed.

The logic that used to reside in the update routine is now distributed across other functions, especially `SCH_Dispatch_Tasks()` and `SCH_Add_Task()`.

Function Descriptions (Summary)

1. `SCH_Init()`

- Initialize all variables and task array elements to their default states (`pTask = 0x0000`, `Delay = 0`, `Period = 0`, `RunMe = 0`).
- Optionally initialize an auxiliary array such as `MARKING[]` to represent the delay relationships among tasks.

2. `SCH_Add_Task()`

- Insert a new task into the list while maintaining the ascending order of `Delay` (so that the task with the shortest delay always appears first).
- After insertion, shift the array elements accordingly and update the `MARKING[]` array to reflect the new delay distribution.

3. `SCH_Delete_Task()`

- Remove a task at the specified index.
- After deletion, shift the remaining elements to fill the gap and refresh the `MARKING[]` array.

4. `SCH_Update()` (**O(1) Version**)

- Each timer cycle (e.g., every 10 ms), operate only on the first task:
 - If `Delay > 0`: decrement `Delay` by 1.
 - If `Delay == 0`: increment `RunMe` to mark the task as ready to run.
- Since this routine always accesses a single element, its time complexity is **O(1)** - making it ideal for ISR/timer execution.

5. `SCH_Dispatch_Tasks()` (**Improved Version**)

Because only the first task is updated in `SCH_Update()`, the dispatch function must handle more logic:

- Check if the first task's `RunMe` value is greater than 0. If not, exit immediately.
- Use the `MARKING[]` array to determine:
 - Tasks with `MARKING = 1` (same delay as the first task) should also be marked as ready to run.
 - Remaining tasks should have their `Delay` decreased by the value of `shortestDelay`.
- Execute all tasks with `RunMe > 0`:
 - If `Period == 0`: permanently remove the task.
 - If `Period != 0`: remove and reinsert it with `Delay = Period`.
- After each deletion or insertion, update `MARKING[]` and ensure the list remains sorted.

Note: Because multiple updates, insertions, and deletions can occur during task dispatching, the worst-case time complexity may reach **O(n²)**.

7.1 SCH_Init

The SCH_Init() function is used to initialize all elements in the task array and the auxiliary array MARKING. The MARKING array will later be discussed in more detail, but in short, it helps track the relative delay relationships among tasks.

```
1  /* ===== GLOBAL VARIABLES ===== */
2  // Array of all tasks, sorted by increasing Delay
3  sTask SCH_tasks_G[SCH_MAX_TASKS];
4  // System error code (0 = no error)
5  uint8_t Error_code_G = 0;
6
7  // Marks tasks with the same Delay as task[0]
8  uint8_t MARKING[SCH_MAX_TASKS];
9
10 // Number of currently active tasks
11 uint32_t task_count = 0;
12 // Ticks elapsed since last dispatch
13 uint32_t elapsed_time = 0;
14
15 /* ===== PRIVATE FUNCTION ===== */
16 static void SCH_Update_Marking(void); // Update MARKING
17
18 /**
19  * SCH_Init - Initialize scheduler
20  * Clear all tasks and reset variables
21  * Call in main() before adding any tasks
22  */
23 void SCH_Init(void) {
24     // Clear all tasks in array
25     for (uint32_t i = 0; i < SCH_MAX_TASKS; i++) {
26         SCH_tasks_G[i].pTask = 0x0000;
27         SCH_tasks_G[i].Delay = 0;
28         SCH_tasks_G[i].Period = 0;
29         SCH_tasks_G[i].RunMe = 0;
30         SCH_tasks_G[i].TaskID = 0;
31         MARKING[i] = 0;
32     }
```

```
33 // Reset counters
34 task_count = 0;
35 elapsed_time = 0;
36 Error_code_G = 0;
37 }
```

This function ensures that the scheduler starts from a clean, empty, and stable state:

- No task is currently assigned (`pTask = 0x0000`);
- All Delay, Period, and RunMe values are set to 0;
- The MARKING array is also initialized to 0, meaning that no delay relationships are established yet.

7.2 SCH_Add_Task

The purpose of the `SCH_Add_Task()` function is to insert a new task into the scheduler's task list while ensuring that the list remains sorted in ascending order of Delay values.

Maintaining this order allows the `SCH_Update()` function to process only the task(s) at the beginning of the list (those with the smallest delay), achieving an $O(1)$ time complexity per timer tick.

The process of adding a new task involves four main steps, described in detail below.

Step 1: Determine the Correct Insertion Position

Assume that the new task has a delay value of x . We must find the index i in the task array such that:

- All tasks on the left side (from 0 to $i - 1$) have $\text{Delay} \leq x$;
- All tasks on the right side (from i onward) have $\text{Delay} > x$.

This condition guarantees that the list remains sorted in ascending order after insertion. If multiple tasks have the same delay value ($\text{Delay} = x$), the new task is placed **after the last task** in that group to preserve relative order.

Step 2: Shift the Array to the Right

After locating the correct index i , we must create space for the new task. All elements from position i to the end of the list are shifted one position to the right.

This ensures that no existing data is lost and that the task list remains contiguous. However, this step may cause the `SCH_Add_Task()` function to have a worst-case time complexity of $O(n)$ (if all elements need to be shifted).

Step 3: Assign Data for the New Task

Once space has been created, assign the following information at index i :

```
SCH_tasks_G[i].pTask = pFunction; // Pointer to the task function
SCH_tasks_G[i].Delay = Delay;      // Initial delay before execution
SCH_tasks_G[i].Period = Period;    // Periodic interval (0 if one-shot)
SCH_tasks_G[i].RunMe = 0;          // Reset run counter
```

This ensures that the new task is correctly inserted and the list order by delay is maintained.

Step 4: Update the MARKING Array

After insertion, the **MARKING** array must be updated. This auxiliary array marks all tasks that share the shortest delay - information crucial for the scheduler, as it allows **SCH_Update()** to trigger only those tasks at the head of the list.

The update rules are as follows:

- The task with the smallest delay (at index 0) always has **MARKING[0] = 1**.
- All other tasks with the same delay as the first task are also marked: **MARKING[i] = 1**.
- Remaining tasks are marked with **MARKING[i] = 0**.

This allows the scheduler to efficiently identify and activate all tasks that share the shortest delay simultaneously.

Example Illustration

Suppose the current list of tasks has the following delay values:

[5, 10, 10, 20, 30, 40]

We want to insert a new task with **Delay = 15**.

- **Step 1 - Find insertion position:** The correct position is after the tasks with delay 10 and before the task with delay 20. Therefore, the insertion index is $i = 3$.
- **Step 2 - Shift array:** Shift all tasks from index 3 onward one position to the right to make space:

[5, 10, 10, _, 20, 30, 40]

- **Step 3 - Insert new task:** Place the new task (**Delay = 15**) at index 3:

[5, 10, 10, 15, 20, 30, 40]

The list remains sorted in ascending order.

- **Step 4 - Update MARKING:** Since the smallest delay is 5, only the first task (and any others with the same delay) are marked as 1:

MARKING = [1, 0, 0, 0, 0, 0, 0]

If multiple tasks have delay 5 (e.g., [5, 5, 10, 15, 20]), the marking becomes:

MARKING = [1, 1, 0, 0, 0, 0]

As a result, the scheduler maintains a correctly ordered list and can efficiently determine which tasks to execute next.


```
1  /**
2   * SCH_Add_Task - Add task to scheduler
3   * Array is always sorted by Delay (ascending order)
4   *
5   * @param pFunction: Pointer to task function
6   * @param DELAY: Number of ticks before first run
7   * @param PERIOD: Number of ticks between runs (0 = one-shot)
8   * @return: Task index in array
9   *
10  * Time Complexity: O(n) - need to find insert position and shift
    array
11  */
12  uint32_t SCH_Add_Task(void (*pFunction)(), uint32_t DELAY, uint32_t
    PERIOD) {
13      /* CASE 1: First task (empty array) */
14      if (task_counts == 0) {
15          SCH_tasks_G[0].pTask = pFunction;
16          SCH_tasks_G[0].Delay = DELAY;
17          SCH_tasks_G[0].Period = PERIOD;
18          SCH_tasks_G[0].RunMe = 0;
19          SCH_tasks_G[0].TaskID = 0;
20          MARKING[0] = 1;
21          elapsed_time = 0;
22          task_count++;
23          return 0;
24      }
25
26      /* CASE 2: Array is full */
27      if (task_count >= SCH_MAX_TASKS) {
28          Error_code_G = ERROR_SCH_TOO_MANY_TASKS;
29          return SCH_MAX_TASKS;
30
31      /* CASE 3: Find insert position (Insertion Sort) */
32      uint32_t insert_index = 0;
33
34
```

```
35 // Find position to keep array sorted by Delay
36 for (uint32_t i = 0; i < task_count; i++) {
37     if (DELAY >= SCH_tasks_G[i].Delay) {
38         insert_index = i + 1;
39     } else {
40         break;
41     }
42 }
43
44 // Shift tasks from insert position to right
45 for (int32_t j = task_count; j > insert_index; j--) {
46     SCH_tasks_G[j] = SCH_tasks_G[j - 1];
47 }
48 // Insert new task at found position
49 SCH_tasks_G[insert_index].pTask = pFunction;
50 SCH_tasks_G[insert_index].Delay = DELAY;
51 SCH_tasks_G[insert_index].Period = PERIOD;
52 SCH_tasks_G[insert_index].RunMe = 0;
53 SCH_tasks_G[insert_index].TaskID = insert_index;
54
55 task_count++;
56
57 // Update MARKING array
58 SCH_Update_Marking();
59
60 return insert_index;
61 }
```

```
71 /**
72  * =====
73  * FUNCTION: SCH_Update_Marking (PRIVATE)
74  * =====
75  * DESCRIPTION: Updates the MARKING array - marks which tasks have
76  *               the same delay as task[0]
77  *
78  * EXAMPLE:
79  *   Task[0]: Delay=10 - MARKING[0]=1
80  *   Task[1]: Delay=10 - MARKING[1]=1 (same delay as task[0])
81  *   Task[2]: Delay=20 - MARKING[2]=0
82  *   Task[3]: Delay=30 - MARKING[3]=0
83  *
84  *   Result: MARKING = [1, 1, 0, 0]
85  *   The first 2 tasks will be ready at the same time!
86  * =====
87  */
88 static void SCH_Update_Marking(void) {
89     if (task_count == 0) return;
90
91     // Get the delay of the first task as reference
92     uint32_t firstDelay = SCH_tasks_G[0].Delay;
93
94     // Iterate through all tasks and mark
95     for (uint32_t n = 0; n < task_count; n++) {
96         if (SCH_tasks_G[n].Delay == first_delay) {
97             MARKING[n] = 1; // same delay as first task
98         } else {
99             MARKING[n] = 0; // different delay
100         }
101     }
```

7.3 SCH_Delete_Task

The purpose of the `SCH_Delete_Task()` function is to remove a specific task from the scheduler's task list while maintaining the list in an increment (ascending) order based on `Delay`. After deletion, all remaining tasks must remain in the correct sequence without leaving gaps in memory.

Implementation Steps

The process of deleting a task involves two main steps:

- **Step 1 - Delete the task at the specified index**

Using the argument `TASK_INDEX`, the function locates the task to be removed. All fields of this task are reset to their initial values as shown below:

```
SCH_tasks_G[TASK_INDEX].pTask = 0x0000;  
SCH_tasks_G[TASK_INDEX].Delay = 0;  
SCH_tasks_G[TASK_INDEX].Period = 0;  
SCH_tasks_G[TASK_INDEX].RunMe = 0;  
MARKING[TASK_INDEX] = 0;
```

This effectively clears the task from the scheduler list.

- **Step 2 - Shift the array to the left**

After deleting a task, all subsequent tasks (from `TASK_INDEX + 1` to the end of the list) are shifted one position to the left. This prevents empty entries in the middle of the task list and preserves the increasing order of delays.

For example, assume the current list of tasks is:

$$[TASK_X, TASK_Y, TASK_Z, TASK_N]$$

If we delete the task at index 1 (`TASK_Y`), the updated list becomes:

$$[TASK_X, TASK_Z, TASK_N]$$

The task list remains contiguous and correctly ordered after this operation.

Result

After completing these two steps, the scheduler retains an incremented list of tasks sorted by delay. No empty slots are left, and all task relationships remain consistent for the next scheduling cycle.

```
1 /**  
2  * SCH_Delete_Task - Remove task from scheduler  
3  * Time Complexity: O(n) - need to shift array left  
4  */  
5 uint8_t SCH_Delete_Task(const uint32_t TASK_INDEX) {  
6     if (TASK_INDEX >= task_count || task_count == 0) {  
7         Error_code_G = ERROR_SCH_CANNOT_DELETE_TASK;
```

```
8         return RETURN_ERROR;
9     }
10    /* CASE 1: Only one task left */
11    if (task_count == 1) {
12        SCH_tasks_G[0].pTask = 0x0000;
13        SCH_tasks_G[0].Delay = 0;
14        SCH_tasks_G[0].Period = 0;
15        SCH_tasks_G[0].RunMe = 0;
16        SCH_tasks_G[0].TaskID = 0;
17        MARKING[0] = 0;
18        task_count = 0;
19    }
20    /* CASE 2: Multiple tasks - shift left */
21    else {
22        // Shift all tasks after deleted task to left
23        for (uint32_t k = TASK_INDEX; k < task_count - 1; k++) {
24            SCH_tasks_G[k] = SCH_tasks_G[k + 1];
25        }
26        // Clear last position
27        uint32_t last_pos = task_count - 1;
28        SCH_tasks_G[last_pos].pTask = 0x0000;
29        SCH_tasks_G[last_pos].Delay = 0;
30        SCH_tasks_G[last_pos].Period = 0;
31        SCH_tasks_G[last_pos].RunMe = 0;
32        SCH_tasks_G[last_pos].TaskID = 0;
33        MARKING[last_pos] = 0;
34
35        task_count--;
36
37        // Update MARKING array
38        if (task_count > 0) {
39            SCH_Update_Marking();
40        }
41    }
42    return RETURN_NORMAL;
43 }
```

7.4 SCH_Update

In the earlier version of the scheduler, the `SCH_Update()` function had a time complexity of $O(n)$. In other words, every time this function was invoked (typically every **10 ms** via a timer interrupt), it had to iterate through the entire list of scheduled tasks and decrement the **Delay** value of each one.

Although this method was simple and easy to implement, it became inefficient as the number of tasks increased. In a real-time system, if the execution time of `SCH_Update()` ever exceeds the timer period (for example, 10 ms), the next interrupt will be delayed - resulting in a **real-time violation**, a critical fault in embedded system design.

Optimization Concept

To overcome this limitation, `SCH_Update()` has been redesigned to reduce its time complexity from $O(n)$ to $O(1)$.

Instead of updating every task in the queue, the function now processes only the **first task**, which always has the **shortest remaining delay**. The operation can be described as follows:

- If the first task is **not yet due** (`Delay > 0`), the function simply decreases its delay by one unit (equivalent to one scheduler tick, e.g., 10 ms).
- If the first task **has reached zero delay** (`Delay == 0`), the function sets the `RunMe` flag to 1, indicating that this task is ready for execution in `SCH_Dispatch_Tasks()`.

With this improvement, the execution time of `SCH_Update()` becomes independent of the number of active tasks, ensuring that the system remains stable and strictly time-accurate even when dozens of tasks are running simultaneously.

A Rational Trade-off

Simplifying `SCH_Update()` means that part of its workload is shifted to the `SCH_Dispatch_Tasks()` function. In the worst case, this may increase the dispatch routine's time complexity to $O(n^2)$ because it must recalculate delays for the remaining tasks based on elapsed time.

However, this trade-off is intentional and well-justified:

- `SCH_Update()` runs **inside the timer interrupt**, where execution time must remain short and deterministic.
- `SCH_Dispatch_Tasks()` runs **in the main program loop**, outside the interrupt context, where longer processing time is acceptable.

In essence, this design adheres to a fundamental principle of embedded systems:

“Always guarantee stability inside the interrupt, even at the cost of extra processing outside it.”

The Role of variable `elapsed_time`

A key element of this mechanism is the variable `elapsed_time`. It acts as an auxiliary timer that tracks the amount of real time that has elapsed since the last task execution.

For example, suppose the first task has an initial delay of 10 ticks (meaning it should run after $10 \times 10 \text{ ms} = 100 \text{ ms}$). At the start:

`elapsed_time = 0`

After each timer tick (10 ms), the values change as follows:

Tick	Time (ms)	Delay	elapsed_time
1	10	9	1
2	20	8	2
3	30	7	3
4	40	6	4
5	50	5	5
$\vdots$	$\vdots$	$\vdots$	$\vdots$
10	100	0	10

When the first task becomes ready (`RunMe = 1`), the variable `shortestDelayTask` now represents the exact amount of time (10 ticks = 100 ms).

The function `SCH_Dispatch_Tasks()` then uses this value to adjust the delay values of all remaining tasks accordingly, ensuring that every task in the system remains synchronized to the same global time base.

Visual Analogy

You can imagine each task as a small countdown clock. Each clock has a hand indicating its remaining delay before activation.

`SCH_Update()` watches only the earliest clock - the one set to ring first - and moves its hand one step every 10 ms. At the same time, it records how much time has passed in `elapsed_time`.

When that first clock rings (i.e., `Delay = 0`), the scheduler knows exactly how much real time has passed and can adjust the remaining clocks accordingly within the dispatch phase.

This approach creates a balance between **simplicity**, **efficiency**, and **precision**, maintaining perfect time synchronization across all tasks in the system.

```
1  /**
2   * SCH_Update - Update function called in TIMER INTERRUPT
3   *
4   * Time Complexity: O(1) - KEY OPTIMIZATION!
5   *
6   * How it works:
7   *   1. ONLY decrement delay of first task (earliest task)
8   *   2. Increment elapsed time counter
9   *   3. If first task is ready - set RunMe flag
10  *
11  * Called from:
12  *   void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
```

```
13 *         if (htim->Instance == TIM2) {
14 *             SCH_Update();
15 *         }
16 *     }
17 */
18 void SCH_Update(void) {
19     if (task_count > 0) {
20         // Step 1: ONLY decrement delay of first task
21         if (SCH_tasks_G[0].Delay > 0) {
22             SCH_tasks_G[0].Delay--;
23         }
24
25         // Step 2: Count elapsed time (used in Dispatch)
26         elapsed_time++;
27
28         // Step 3: If first task is ready - set flag
29         if (SCH_tasks_G[0].Delay == 0) {
30             SCH_tasks_G[0].RunMe++;
31         }
32     }
33 }
```

7.5 SCH_Dispatch_Tasks

Purpose

The function `SCH_Dispatch_Tasks()` is responsible for executing tasks that are ready to run (`RunMe > 0`) and updating the remaining Delay values for all other tasks in the queue. In the improved scheduler design, `SCH_Update()` only decrements the Delay of the first task to ensure deterministic timing within the ISR. Therefore, all remaining Delay adjustments and task scheduling are handled by the Dispatch function, maintaining accurate timing synchronization across all tasks.

Function Objectives

1. Check if the first task in the queue (`task[0]`) is ready (`RunMe > 0`).
2. Update Delay and RunMe values for other tasks according to the `MARKING[]` array, which indicates tasks sharing the same Delay as the first task.
3. Execute all tasks that are ready (`RunMe > 0`).
4. Post-execution handling:

- One-shot tasks (`Period == 0`) → remove from the task list.
- Periodic tasks (`Period != 0`) → remove and re-add with `Delay = Period`. After each addition or removal, `MARKING[]` must be updated accordingly.

Design Principles

- `SCH_Update()` runs in the ISR and only processes the first task → guarantees **O(1)** runtime.
- `SCH_Dispatch_Tasks()` runs in the main loop (outside ISR) and compensates for remaining updates:
 - Decreases `Delay` for tasks not in the same group (`MARKING == 0`) by `elapsed_time`.
 - Sets `RunMe = 1` for tasks in the same group (`MARKING == 1`).
- `MARKING[]` defines the relation "same Delay as `task[0]`" and only needs updating when tasks are added or deleted.

Algorithm Details

1. **Check starting condition:** If `task_count == 0` or `SCH_tasks_G[0].RunMe == 0` → exit.

2. **Update Delay and RunMe based on MARKING and elapsed_time:**

- For tasks in the same group (`MARKING[i] == 1`):
`SCH_tasks_G[i].Delay = 0`
`SCH_tasks_G[i].RunMe = 1`
- For tasks in a different group (`MARKING[i] == 0`):
if `SCH_tasks_G[i].Delay >= elapsed_time`:
 `SCH_tasks_G[i].Delay -= elapsed_time`
else:
 `SCH_tasks_G[i].Delay = 0`

3. **Execute ready tasks (while loop):**

```
while SCH_tasks_G[0].RunMe > 0:
    (*SCH_tasks_G[0].pTask)();    // Execute task if not NULL
    SCH_tasks_G[0].RunMe--;

    if Period == 0:                // one-shot
        SCH_Delete_Task(0);
    else:                          // periodic
        *temp_func = pTask;
        temp_period = Period;
        SCH_Delete_Task(0);
        SCH_Add_Task(temp_func, temp_period, temp_period);
    SCH_Update_Marking();          // update MARKING[] after each add/delete
```

Repeat until `RunMe` of the first task becomes 0.

4. **Reset counter:** Once all ready tasks are processed, reset `elapsed_time = 0` for the next `SCH_Update()` cycle.

Illustrative Example

Assume the following tasks are added:

Task	Delay	Period
TASK0	10	0
TASK1	10	10
TASK2	20	20
TASK3	20	20
TASK4	30	30
TASK5	40	0

Bảng 1: *Initial Task Configuration*

Initial state:

$DELAY_LIST = [10, 10, 20, 20, 30, 40]$, $MARKING = [1, 1, 0, 0, 0, 0]$, $elapsed_time = 0$

After 100 ms (10 ticks):

$DELAY_LIST = [0, 10, 20, 20, 30, 40]$, $elapsed_time = 10$

Dispatch Execution:

1. Check `RunMe`: `task[0]` (TASK0) $\rightarrow$ `RunMe` > 0
2. Update Delay & `RunMe` using `MARKING`:

Same group (`MARKING` = 1) : $Delay = 0, RunMe = 1$

Different group (`MARKING` = 0) : $20 \rightarrow 10, 20 \rightarrow 10, 30 \rightarrow 20, 40 \rightarrow 30$

3. Result:

$DELAY_LIST = [0, 0, 10, 10, 20, 30]$, $RUNME = [1, 1, 0, 0, 0, 0]$, $MARKING = [1, 1, 0, 0, 0, 0]$

4. Execute tasks according to `RunMe`:

- TASK0 (one-shot) $\rightarrow$ delete $\rightarrow$ $DELAY_LIST = [0, 10, 10, 20, 30]$
- TASK1 (periodic) $\rightarrow$ delete and re-add with $Delay = Period = 10 \rightarrow$ $DELAY_LIST = [10, 10, 10, 20, 30]$, $MARKING = [1, 1, 1, 0, 0]$

No tasks remain with `RunMe` $> 0 \rightarrow$ loop ends and reset `elapsed_time = 0`.

Outcome: Dispatch executed the necessary tasks, updated remaining Delays, and maintained synchronization of all tasks using `elapsed_time`.

```
1  /**
2  * SCH_Dispatch_Tasks - Execute ready tasks - CALLED IN MAIN LOOP
3  * Time Complexity:  $O(n^2)$  in worst case
4  *
5  * How it works (2 STEPS):
6  *   STEP 1: UPDATE DELAY FOR ALL TASKS
7  *     - Tasks with MARKING=1: Set Delay=0, RunMe=1
8  *     - Tasks with MARKING=0: Subtract elapsed time
9  *
10 *   STEP 2: EXECUTE READY TASKS
11 *     - Call task function
12 *     - If one-shot (Period=0): Delete task
13 *     - If periodic: Delete then re-add with new delay
14 *
15 */
16 void SCH_Dispatch_Tasks(void) {
17     // Check if any task is ready
18     if (task_count > 0 && SCH_tasks_G[0].RunMe > 0) {
19
20         /* STEP 1: UPDATE DELAY FOR ALL TASKS */
21         for (uint32_t m = 0; m < task_count; m++) {
22             if (MARKING[m] == 0) {
23                 // Task has DIFFERENT delay from task[0]
24                 // Subtract elapsed time
25                 if (SCH_tasks_G[m].Delay >= elapsed_time) {
26                     SCH_tasks_G[m].Delay -= elapsed_time;
27                 } else {
28                     SCH_tasks_G[m].Delay = 0;
29                 }
30             } else {
31                 // Task has SAME delay as task[0]
32                 // Also ready to run!
33                 SCH_tasks_G[m].Delay = 0;
34                 SCH_tasks_G[m].RunMe = 1;
35             }
36         }
37     }
```

```
37      /* STEP 2: EXECUTE READY TASKS */
38      while (task_count > 0 && SCH_tasks_G[0].RunMe > 0) {
39          // 1. RUN TASK
40          if (SCH_tasks_G[0].pTask != 0x0000) {
41              (*SCH_tasks_G[0].pTask)(); // Call task function
42          }
43          // 2. CLEAR FLAG
44          SCH_tasks_G[0].RunMe--;
45          // 3. HANDLE TASK BASED ON PERIOD
46          if (SCH_tasks_G[0].Period == 0) {
47              /* ONE-SHOT TASK: Run once - DELETE */
48              SCH_Delete_Task(0);
49          } else {
50              /* PERIODIC TASK: Repeat - DELETE then RE-ADD */
51
52              // Save task info
53              void (*temp_func)(void) = SCH_tasks_G[0].pTask;
54              uint32_t temp_period = SCH_tasks_G[0].Period;
55
56              // Delete old task
57              SCH_Delete_Task(0);
58
59              // Re-add task with delay = period
60              SCH_Add_Task(temp_func, temp_period, temp_period);
61          }
62      }
63
64      // Reset elapsed time counter
65      elapsed_time = 0;
66  }
67  // SCH_Report_Status();
68  // SCH_Go_To_Sleep();
69 }
```



Tài liệu

[1]